Are you making good use of your compute?

Three Stages of vLLM Inference Cluster Optimization

Li Mengxuan | Co-founder & CTO, Dynamia | HAMi Author

- Phase 1: vLLM as a workload
- Phase 2: vLLM + kvcache/PD Disaggregation
- Phase 3: vLLM + Kvcache + PD Disaggregation + GPU Virtualization

Phase 0: vLLM as a process

Use case: You have a GPU server & want to validate vLLM inference as fast as possible

Phase 0: vLLM as a process

Use case: You have a GPU server & want to validate vLLM inference as fast as possible

With Conda

```
conda create -n vllm python=3.11 -y
conda activate vllm

# CUDA 12.1
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121

CUDA_VISIBLE_DEVICES=0,1,2,3 python -m
vllm.entrypoints.openai.api_server \
  --model meta-llama/Meta-Llama-3-70B-Instruct \
  --host 0.0.0.0 \
  --port 8000 \
  --tensor-parallel-size 4 \
  --gpu-memory-utilization 0.9
```

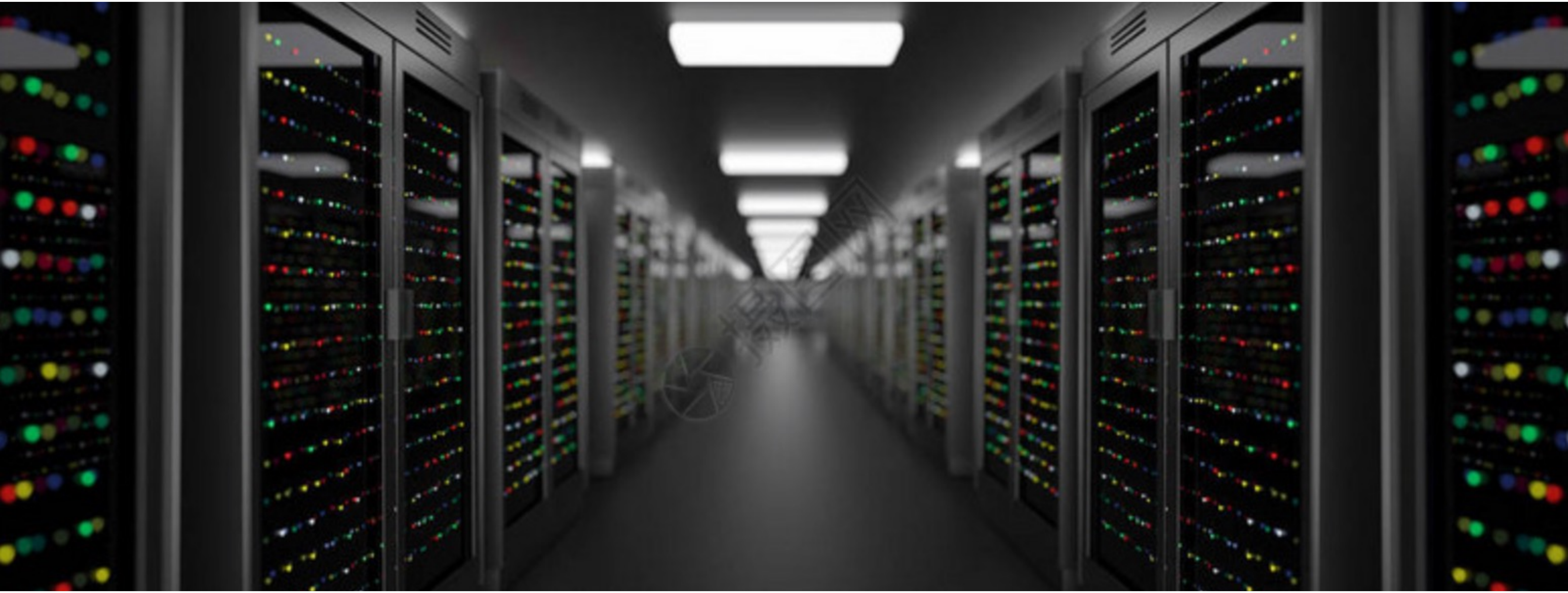
With Docker

```
docker run --runtime nvidia --gpus '"device=0,1,2,3"' \
  -v ~/.cache/huggingface:/root/.cache/huggingface \
  -e HUGGING_FACE_HUB_TOKEN=hf_xxxxxxxxxxxx \
  -p 8000:8000 \
  --ipc=host \
  vllm/vllm-openai:latest \
  --model meta-llama/Meta-Llama-3-8B-Instruct \
  --tensor-parallel-size 1 \
  --gpu-memory-utilization 0.9
```

Only suitable for dev/test clusters

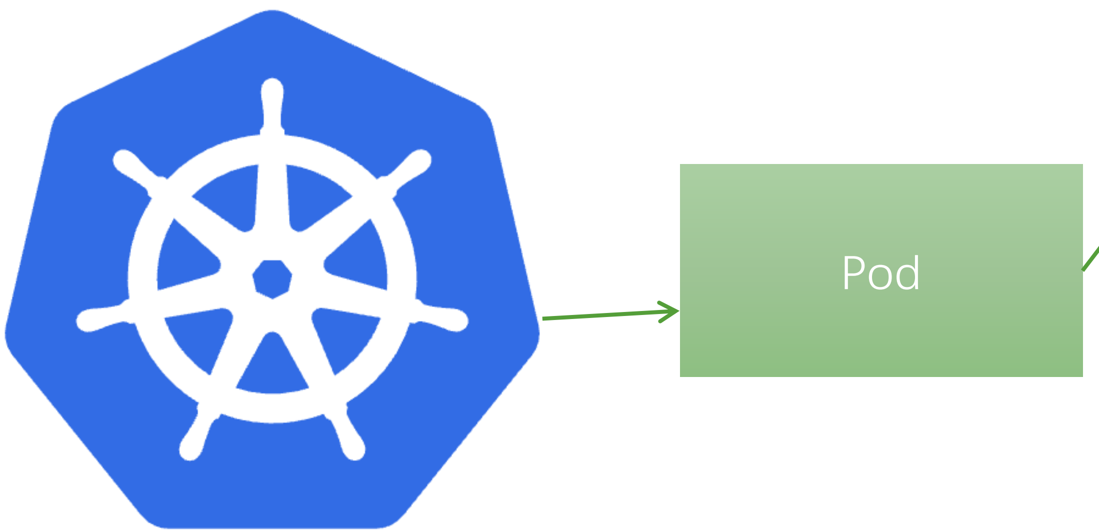
Phase 1: vLLM as a workload

Use case: You want to schedule vLLM workloads across a GPU cluster



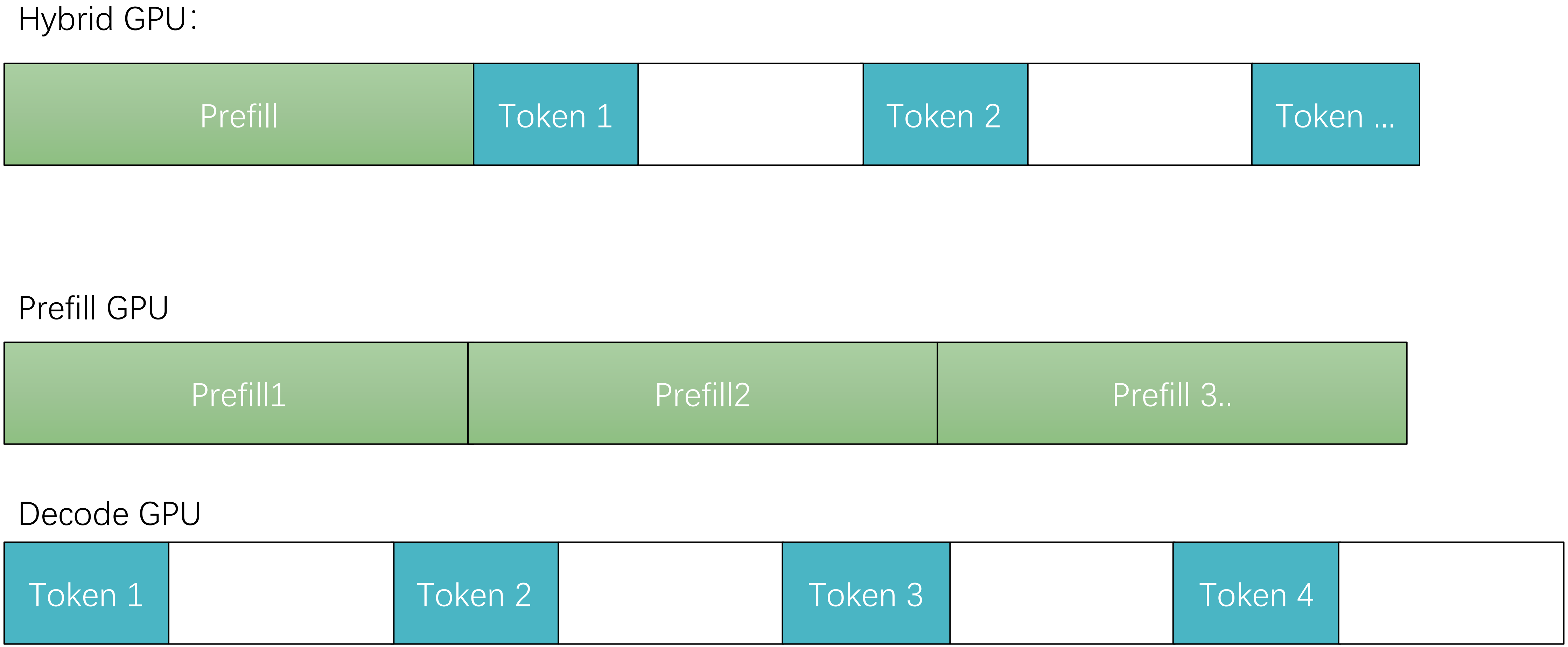
Phase 1: vLLM as a workload

Use case: You want to schedule vLLM workloads across a GPU cluster




```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-server
  namespace: vllm-inference
  labels:
    app: vllm
spec:
    '''
      containers:
      - name: vllm
        image: vllm/vllm-openai:latest
        args:
        - --model
        - meta-llama/Meta-Llama-3-8B-Instruct
        - --host
        - "0.0.0.0"
        - --port
        - "8000"
        - --gpu-memory-utilization
        - "0.9"
        - --max-model-len
        - "8192"
        resources:
          limits:
            nvidia.com/gpu: 1
            memory: "32Gi"
            cpu: "8"
```

Phase 2: vLLM + PD Disaggregation



Phase 2: vLLM + PD Disaggregation

Prefill

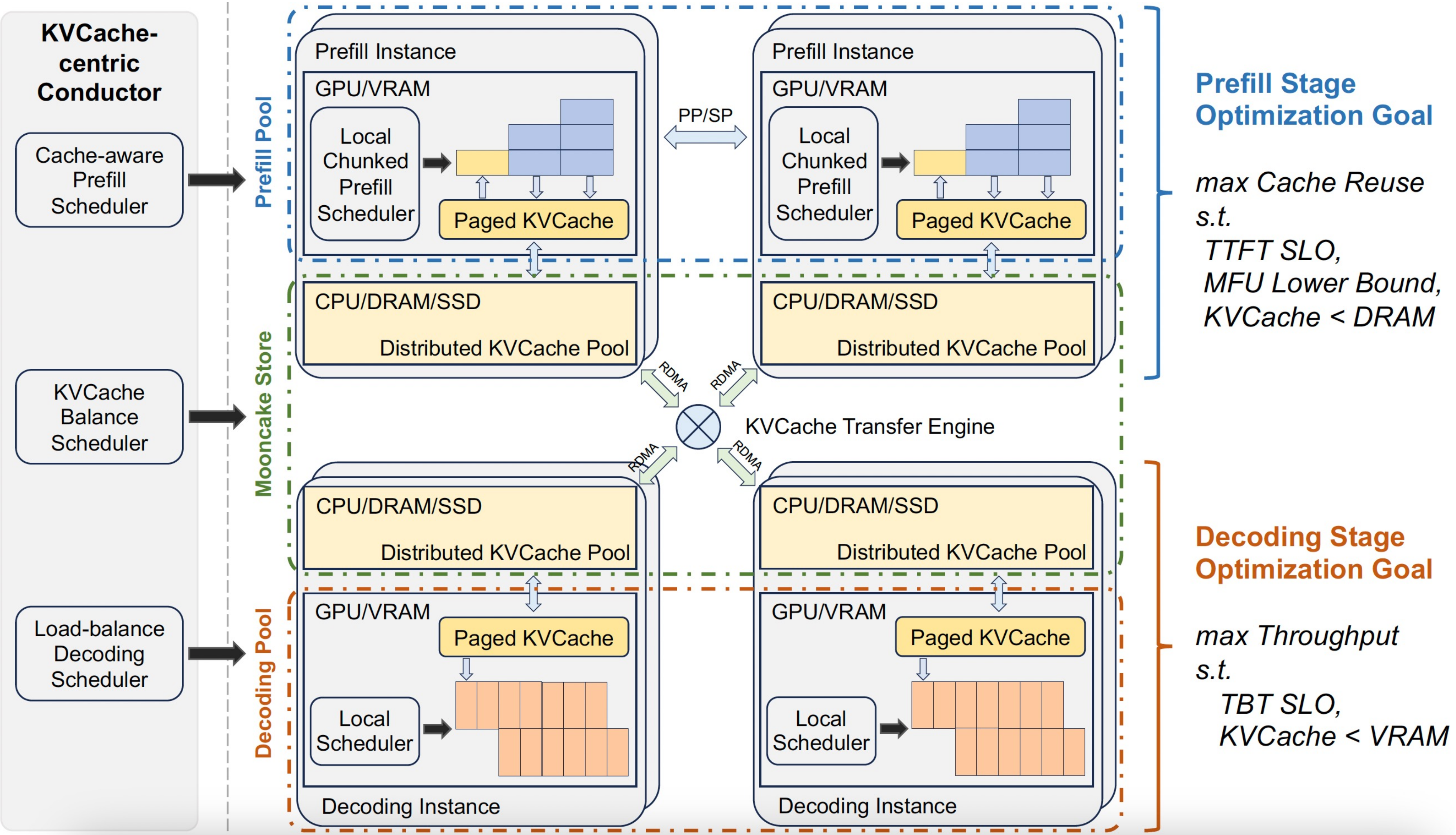
```
CUDA_VISIBLE_DEVICES=0 \
NCCL_SOCKET_IFNAME=eth0 \
NCCL_DEBUG=INFO \
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
  --port 8100 \
  --enforce-eager \
  --kv-transfer-config '{
   "kv_connector": "P2pNcclConnector",
   "kv_role": "kv_producer",
   "kv_rank": 0,
   "kv_parallel_size": 2
  }'
```

P2pNcclConnector

Decode

```
CUDA_VISIBLE_DEVICES=0 \
NCCL_SOCKET_IFNAME=eth0 \
NCCL_DEBUG=INFO \
MASTER_ADDR=192.168.1.10 \
MASTER_PORT=29500 \
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
  --port 8200 \
  --enforce-eager \
  --kv-transfer-config '{
   "kv_connector": "P2pNcclConnector",
   "kv_role": "kv_consumer",
   "kv_rank": 1,
   "kv_parallel_size": 2
  }'
```

Phase 2: vLLM + PD Disaggregation: Mooncake



Phase 2: vLLM + PD Disaggregation: Mooncake

Conductor

```
python -m mooncake.conductor \
--transfer-engines
192.168.1.10:50051,192.168.1.11:50
051 \
  --host 0.0.0.0 \
  --port 9000 \
  --policy kvcache-locality
```

Prefill

```
python -m
mooncake.transfer_engine \
  --host 0.0.0.0 \
  --port 50051 \
  --role prefill
```

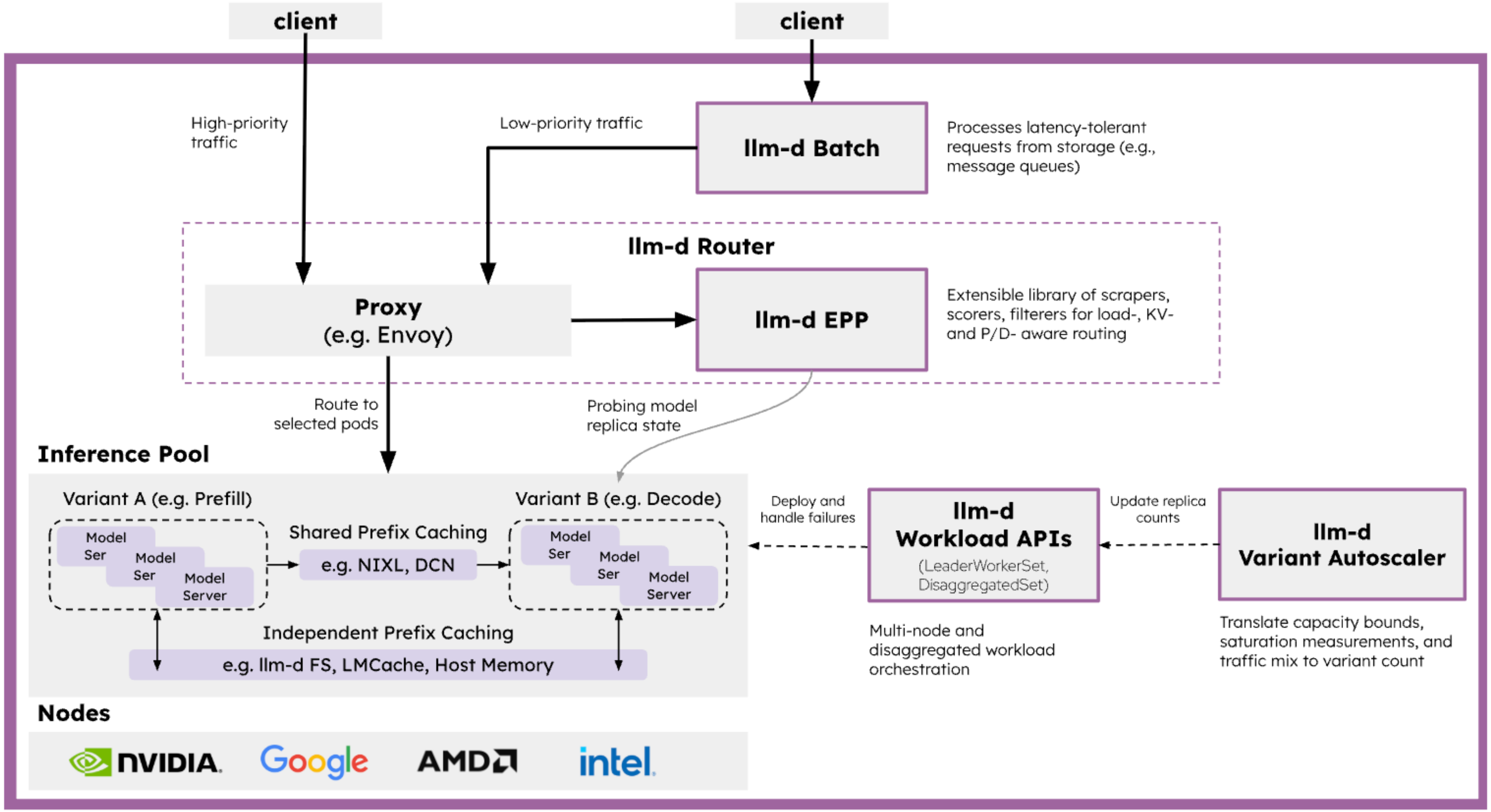
```
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Meta-Llama-3-8B-Instruct \
  --host 0.0.0.0 \
  --port 8100 \
  --tensor-parallel-size 1 \
  --gpu-memory-utilization 0.9 \
  --kv-transfer-config
'{"kv_connector":"MooncakeConnector","kv_role":"kv_
producer","kv_rank":0,"kv_parallel_size":1,"kv_buffer_size"
:5e9}'
```

Decode

```
python -m
mooncake.transfer_engine \
  --host 0.0.0.0 \
  --port 50051 \
  --role decode
```

```
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Meta-Llama-3-8B-Instruct \
  --host 0.0.0.0 \
  --port 8200 \
  --tensor-parallel-size 1 \
  --gpu-memory-utilization 0.9 \
  --kv-transfer-config
'{"kv_connector":"MooncakeConnector","kv_role":"kv_co
nsumer","kv_rank":0,"kv_parallel_size":1,"kv_buffer_size":5e9}
```

Phase 2: vLLM + PD Disaggregation: LLM-D



Phase 2: vLLM + PD Disaggregation: LLM-D

Inference Pool

EPP

```yaml
spec:
  schedulingProfiles:
  - name: pd-disaggregated
    plugins:
    - type: pd-profile-handler
      parameters:
        prefillPool: llama3-prefill-pool
        decodePool: llama3-decode-pool
```

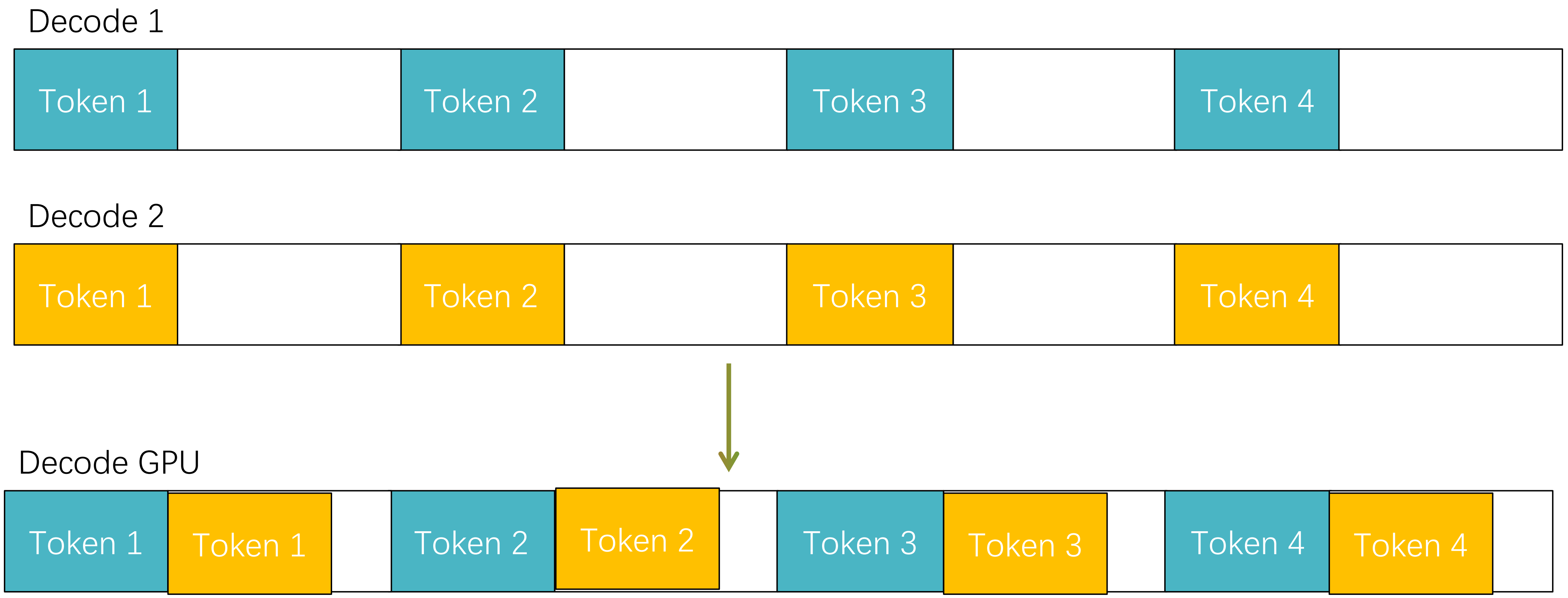
```yaml
# prefill pool
apiVersion: gateway.llm-d.ai/v1alpha1
kind: InferencePool
metadata:
  name: llama3-prefill-pool
spec:
  selector:
    app: vllm-llama3
    role: prefill
  targetPort: 8000

---
# decode pool
apiVersion: gateway.llm-d.ai/v1alpha1
kind: InferencePool
metadata:
  name: llama3-decode-pool
spec:
  selector:
    app: vllm-llama3
    role: decode
  targetPort: 8000
```

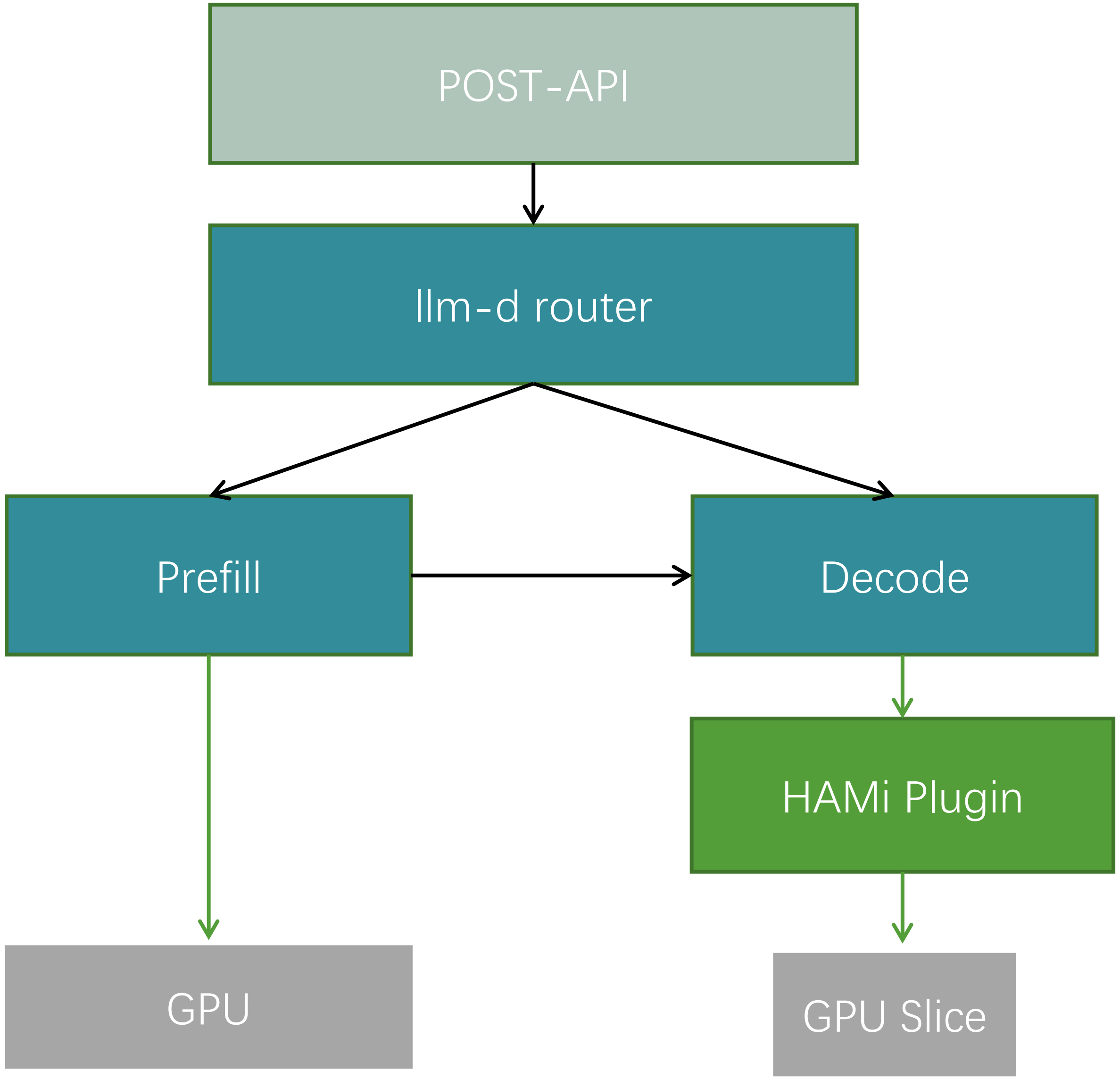
vLLM Deployment for Prefill

vLLM Deployment for Decode

Phase 3: vLLM + PD Disaggregation + GPU Virtualization



Phase 3: vLLM + PD Disaggregation + GPU Virtualization



Phase 3: vLLM + PD Disaggregation + GPU Virtualization

Inference Pool

```
# prefill pool
apiVersion: gateway.llm-d.ai/v1alpha1
kind: InferencePool
metadata:
  name: llama3-prefill-pool
spec:
  selector:
    app: vllm-llama3
    role: prefill
  targetPort: 8000

---
# decode pool
apiVersion: gateway.llm-d.ai/v1alpha1
kind: InferencePool
metadata:
  name: llama3-decode-pool
spec:
  selector:
    app: vllm-llama3
    role: decode
  targetPort: 8000
```

vLLM Deployment for Prefill

```
resources:
  limits:
    nvidia.com/gpu: 1
  requests:
    nvidia.com/gpu: 1
```

vLLM Deployment for Decode

```
resources:
  limits:
    nvidia.com/gpu: 1
    nvidia.com/gpumem-percentage: 50
  requests:
    nvidia.com/gpu: 1
    nvidia.com/gpumem-percentage: 50
```

Thank You
Follow us!

HAMi Community Assistant

Dynamia Official Website

HAMi Official WeChat

Linkedin:
linkedin.com/company/project-hami-io

X:
@HAMiProject

github:
github.com/project-hami/hami